Veermata Jijabai Technological Institute (VJTI), Mumbai

MINI PROJECT REPORT

"CircuitVision: Automated Conversion of Hand-Drawn Circuit Diagrams into Validated, Simulatable SPICE Netlists using Computer Vision, Image Processing and NLP"

Submitted in partial fulfilment of the requirements for the subject

Minor/MDM: AI and ML

Submitted by

Name	Roll Number
Pranshu Kumar	231090054
Pawan Shinde	231090063

Final Year B.Tech, EXTC

Project Guide

Prof. Vedashree Awati

Veermata Jijabai Technological Institute (VJTI), Mumbai

Academic Year 2026–27, Semester VII

Date of submission: [Date of submission]

Certificate

This is to certify that the mini project report entitled **"CircuitVision: Automated Conversion of Hand-Drawn Circuit Diagrams into Validated, Simulatable SPICE Netlists using Computer Vision, Image Processing and NLP"** submitted by **Pranshu Kumar (231090054)** and **Pawan Shinde (231090063)**, students of Final Year B.Tech EXTC, is a record of bona fide work carried out by them under my guidance and supervision, in partial fulfilment of the requirements for **Minor/MDM: AI and ML** at Veermata Jijabai Technological Institute (VJTI), Mumbai, during the Academic Year 2026–27.

_____
Prof. Vedashree Awati
Project Guide

Date: _____
Place: Mumbai

Declaration

We declare that this written submission represents our ideas in our own words, and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated or falsified any idea, data, fact or source in our submission.

All datasets, open-source libraries, published methods and software tools used in this work, including AI-assisted development tools, are acknowledged in this report. Every experimental result reported here is labelled with the kind of data it was measured on, and results that have not yet been obtained are marked as pending rather than estimated.

Name	Roll Number	Signature
Pranshu Kumar	231090054	
Pawan Shinde	231090063	

Date: _____
Place: Mumbai

Acknowledgement

We express our sincere gratitude to our project guide, **Prof. Vedashree Awati**, for her guidance, encouragement and constructive feedback throughout this mini project.

We thank Veermata Jijabai Technological Institute (VJTI), Mumbai, for offering the Minor/MDM programme in AI and ML and for the academic environment that made this work possible.

We are grateful to F. Thoma, J. Bayer, Y. Li, A. Dengel and their colleagues at DFKI for publishing the CGHD dataset of hand-drawn circuit diagrams under an open licence, and to the developers of Ultralytics YOLO, OpenCV, EasyOCR, ngspice, NumPy, Gradio and pytest, on whose work this project is built. We also acknowledge the AI-assisted development tools used while writing and testing the software.

Finally, we thank our families and friends for their constant support.

Pranshu Kumar (231090054)
Pawan Shinde (231090063)

Abstract

A circuit sketched on paper has to be retyped as a SPICE netlist before it can be simulated, and node-numbering mistakes made while retyping are easy to miss. **CircuitVision** takes a phone photograph of a hand-drawn circuit, or an English description of one, and produces a SPICE netlist that is checked against a formal grammar and then simulated.

The image path uses a YOLOv8s detector for nine symbol classes, a HOG and linear-SVM classifier for the polarity of diodes and sources, paper-relative binarisation with Otsu thresholding, scale-adaptive wire tracing with crossover handling, and EasyOCR for component values. The text path uses a hidden Markov model tagger with Viterbi decoding and a series-parallel parser. Both paths share a context-free grammar with an Earley recogniser, semantic checks and ngspice simulation. The detector is set up for the public CGHD dataset with a drafter-wise split (validation drafter 11, test drafter 12).

On synthetic drawings with known circuit graphs, the wire tracer recovers the exact topology in 99.4% of 165 trials across eleven photo conditions, and with ground-truth boxes 18 of 18 circuits match their reference netlists, including the simulated operating point. The text path reaches 90% exact circuit match on a fresh 20-sentence test set and 75% on a second set written by a teammate who had not seen the parser. The dataset has been audited, but YOLOv8s training and testing on CGHD had not finished at the time of writing, so no real-photo accuracy is claimed.

Keywords: *hand-drawn circuit recognition, YOLOv8, image processing, wire tracing, OCR, SPICE netlist, context-free grammar, Earley parser, hidden Markov model, ngspice*

Table of Contents

Certificate ..... **i**

Declaration ..... **ii**

Acknowledgement ..... **iii**

Abstract ..... **iv**

List of Figures ..... **vii**

List of Tables ..... **vii**

List of Abbreviations ..... **vii**

Chapter 1: Introduction **1**

1.1 Background 1

1.2 Problem Statement 1

1.3 Motivation 1

1.4 Objectives 1

1.5 Scope and Applications 2

1.6 Organisation of the Report 2

Chapter 2: Literature Review **2**

2.1 Early and Classical Approaches 2

2.2 Datasets: CGHD 3

2.3 Deep Detection and Graph Extraction 3

2.4 Relevant NLP and Validation Techniques 3

2.5 Comparison of Existing Approaches 3

2.6 Limitations of Existing Solutions 4

2.7 Gap Addressed by this Project 4

Chapter 3: Problem Definition and Proposed System **4**

3.1 Detailed Problem Definition 4

3.2 Proposed AI-Based Solution 5

3.3 System Architecture 5

3.4 Functional Requirements 5

3.5 Non-Functional Requirements 6

3.6 Expected Outcomes 6

Chapter 4: AI Methodology **6**

4.1 AI Techniques and Why They Suit the Problem 6

4.2 Dataset and its Source 7

4.3 Data Preprocessing 8

4.4 Feature Extraction and Representation 9

4.5 Model Selection and Training 10

4.6 System Workflow 11

4.7 Tools, Libraries and Technologies 11

Chapter 5: Implementation **12**

5.1 Software and Hardware Requirements 12

5.2 Module-wise Description 12

5.3 Important Algorithms 12

5.4 Implementation Details 14

5.5 Screenshots of the Running System 15

5.6 Sample Inputs and Outputs 16

Chapter 6: Results and Discussion....16
6.1 Evaluation Design....16
6.2 Evaluation Metrics....17
6.3 Real CGHD Results (baseline_v1)....17
6.4 Wire Tracing on Synthetic Photos (SYNTHETIC)....18
6.5 End-to-End Reconstruction against Reference Netlists (SYNTHETIC)....18
6.6 OCR and Orientation (SYNTHETIC)....19
6.7 Text-to-Netlist Results (HAND-WRITTEN)....19
6.8 Comparison with Baselines....20
6.9 Defects Found during Development....20
6.10 Limitations and Observed Errors....21
6.11 Ethical and Privacy Considerations....22
Chapter 7: Conclusion and Future Scope....22
7.1 Summary of the Work....22
7.2 Objectives Achieved....22
7.3 Key Findings....23
7.4 Current Limitations....23
7.5 Future Improvements....23
References....23
Appendix A: Important Source Code....25
Appendix B: Configuration Details....26
Appendix C: Sample Dataset Record....26
Appendix D: Test Cases....27

List of Figures

Figure 1: System architecture of CircuitVision: image path, text path and shared back end.....5
Figure 2: Six of the eleven synthetic photo conditions applied to the crossover circuit.....8
Figure 3: Binarisation stages on a synthetic photo with grid paper, a shadow, noise and JPEG compression.....9
Figure 4: Rotation augmentation of a synthetic diode crop; the + side follows from the label under the convention fixed at Gate 1 (anode or + on the left at 0°, counter-clockwise).....10
Figure 5: Real-data experiment protocol with two human verification gates.....11
Figure 6: Wire tracing on a synthetic bridge drawn on grid paper: nets coloured, and the generated netlist 13
Figure 7: Gradio web demo: input, detected components and nets, SPICE netlist, validation and DC operating point (synthetic input and model).....15
Figure 8: Command-line image path on the example drawing with ground-truth boxes, validation and simulation.....15
Figure 9: Command-line text path (tags, netlist, validation) and the test suite.....16
Figure 10: Example input drawing and the output overlay produced by the pipeline.....16
Figure 11: Exact-topology rate per photo condition (SYNTHETIC, ground-truth boxes).....18
Figure 12: Detector-mode failure on a synthetic crossover circuit: both crossover symbols are missed, merging two independent loops.....19
Figure 13: Text-to-netlist scores on the four evaluation runs (HAND-WRITTEN).....19

List of Tables

Table 1: Comparison of existing approaches and CircuitVision.....3
Table 2: Functional requirements.....5
Table 3: AI techniques used and why each suits its sub-problem.....6
Table 4: Mapping from CGHD labels to the nine detector classes.....7
Table 5: Fixed baseline training configuration (baseline_v1).....10
Table 6: Tools, libraries and technologies.....11
Table 7: Software and hardware requirements.....12
Table 8: Modules of the circuitvision package.....12
Table 9: Sample text inputs and generated netlists (actual outputs).....16
Table 10: Validator outputs on deliberately faulty netlists (actual outputs).....16
Table 11: Evaluation metrics by stage.....17
Table 12: Real CGHD results on test drafter 12 (REAL CGHD).....17
Table 13: End-to-end results on CGHD-format synthetic data (SYNTHETIC).....18
Table 14: Text-to-netlist results on hand-written evaluation sets.....19
Table 15: Failures on the fresh 20-sentence set.....20
Table 16: Defects found during development and their fixes.....21
Table 17: Status of the objectives.....22
Table 18: Automated regression tests (129 in total).....27

List of Abbreviations

AP	Average Precision

CFG	Context-Free Grammar
CGHD	Circuit Graph Hand-Drawn dataset
CER	Character Error Rate
CV	Computer Vision
EXIF	Exchangeable Image File Format (photo metadata)
GT	Ground Truth
HMM	Hidden Markov Model
HOG	Histogram of Oriented Gradients
IoU	Intersection over Union
mAP	mean Average Precision
NLP	Natural Language Processing
OCR	Optical Character Recognition
RKM	Resistor value code with the multiplier as decimal point (e.g. 4k7)
SPICE	Simulation Program with Integrated Circuit Emphasis
SVM	Support Vector Machine
VOC	PASCAL Visual Object Classes (annotation format)
YOLO	You Only Look Once (object detector)

Chapter 1: Introduction

1.1 Background

Circuits are usually sketched on paper before anyone simulates them. To simulate one, the sketch has to be typed into a simulator such as SPICE [1] as a *netlist*: a list in which every component is named, given a value and connected to numbered nodes. A single wrong node number produces a circuit that simulates without error but is not the one that was drawn.

Automating this step is harder for hand drawings than for printed schematics. Every person draws symbols differently, wires wobble or have gaps, photos are taken in uneven light, often on grid paper, and values are handwritten. The output also has to be right as a *graph*, not just as a list of symbols: if one wire is attached to the wrong node, every node voltage changes, even when every symbol was detected correctly.

Two things made the problem practical for a semester project: single-stage object detectors such as YOLO [2], which can be fine-tuned on a single free cloud GPU, and CGHD [3], a public dataset of photographed hand-drawn circuits with bounding-box annotations.

1.2 Problem Statement

Given a phone photograph of a hand-drawn circuit made of two-terminal elements, produce a SPICE netlist whose circuit graph, component values and source polarity match the drawing, and which can be simulated. Given an English description of a series-parallel circuit, produce the same kind of netlist. In both cases, the netlist must be checked automatically for syntactic and semantic validity before simulation.

1.3 Motivation

- **Retyping is slow and error-prone.** Node-numbering mistakes are common when a sketch is copied into a simulator by hand, and they are hard to spot afterwards.
- **It mixes learned and rule-based methods.** Finding symbols and reading handwriting need learned models; connectivity, the netlist grammar and simulation can be handled exactly, which keeps those parts testable.
- **The output can be checked.** A reconstructed circuit can be compared with the drawn one as a graph and by simulation, which gives a stricter test than detection accuracy alone.
- **It fits the course requirement.** The minor asks for a project grounded in image processing, computer vision or NLP rather than generic AI; this project uses all three.

1.4 Objectives

1. Detect nine circuit-symbol classes in real hand-drawn photos (CGHD) with a reproducible YOLOv8s baseline, evaluated on a held-out drafter.
2. Recover the polarity of diodes and voltage sources from the orientation of their symbols.
3. Trace wires into electrical nets robustly to photo resolution, grid paper, shadows, noise, pen gaps and imprecise detector boxes, including junctions and wire crossovers.
4. Read component values from handwritten labels and attach each value to the correct component.

5. Generate SPICE netlists, validate them with a context-free grammar, an Earley parser and semantic checks, and simulate them with ngspice.
6. Provide a second input path that translates English series-parallel circuit descriptions into netlists using HMM tagging and compositional semantic parsing.
7. Evaluate every stage separately, with ground-truth boxes and with detector boxes, and keep real, synthetic and pending results strictly apart.

1.5 Scope and Applications

In scope. Two-terminal elements (resistors, capacitors, inductors, DC voltage sources and diodes), together with ground symbols, junction dots, wire crossovers and text labels. Simulation is a DC operating point.

Out of scope. Transistors, integrated circuits, op-amps, switches, lamps, AC and current sources, and any element with more than two terminals. Such symbols may still appear in a drawing; they are masked so that they do not corrupt the reconstruction of the rest, but they are not reconstructed.

Applications. Checking homework and laboratory circuits, digitising notebook and whiteboard sketches, and teaching: the overlay shows each net in a different colour, so a student can see which wires the program thinks are connected. The text path allows a circuit to be entered as a sentence.

1.6 Organisation of the Report

Chapter 2 reviews related work. Chapter 3 defines the problem precisely and presents the proposed system. Chapter 4 describes the AI methodology, dataset and training. Chapter 5 covers the implementation with algorithms, screenshots and sample outputs. Chapter 6 presents results and discussion, and Chapter 7 concludes. Source-code excerpts, configuration and test details are in the appendices; the full code is in the project repository [4].

Chapter 2: Literature Review

2.1 Early and Classical Approaches

Off-line recognition of hand-drawn circuits was studied well before deep learning; Edwards and Chandran [5] is one of the early systems. Thoma *et al.* survey this earlier work and point out that each system was evaluated on its own small collection, which is the reason they released a public dataset [3]. Our summary of the older systems relies on that survey.

Moetesum *et al.* [6] segmented components from binarised images with morphological operations, described each component with HOG features [7] and classified it with an SVM [8]. On 100 hand-drawn diagrams containing 350 components they report 87.7% segmentation accuracy and a 92% classification rate. According to the same survey, Patare and Joshi [9] classified hand-drawn logic-gate symbols with Fourier descriptors and an SVM and tested on ten sketches [3]. These systems show that hand-crafted features can classify isolated symbols, but neither recovers how the symbols are connected, which is what a netlist needs.

2.2 Datasets: CGHD

Thoma *et al.* [3] published CGHD, which they describe as the first public ground-truth dataset of hand-drawn circuit images. The first release contained 1,152 images of 144 circuits drawn by 12 drafters, each circuit photographed four times, with 48,563 bounding-box annotations in 45 classes; a Faster R-CNN baseline reached 52% mAP. The dataset has since grown: later papers describe 2,208 images in 58 classes [10] and 2,424 images from 25 drafters [11], and newer releases add rotation, mirror and text annotations [12]. This project uses the versioned Zenodo release 14 [13], pinned by its MD5 checksum for reproducibility.

2.3 Deep Detection and Graph Extraction

Reddy and Panicker [14] detected components with YOLOv5 and located nodes with a Hough-transform method, reporting 98.2% mAP@0.5 for component detection and 80% accuracy for rebuilding the schematic on their own dataset, at about 0.33 s per image. Bayer, Roy and Dengel [10] extended CGHD with binary segmentation maps and polygon and keypoint annotations, and extracted the circuit graph with a Mask R-CNN instance-segmentation model followed by geometric keypoint matching; they describe the result qualitatively as reasonable but incomplete. Bayer, van Waveren and Dengel [11] built a modular pipeline with symbol and text detection (Faster R-CNN, 18% mAP on validation drafters 21–22), U-Net stroke segmentation (98.16% pixel accuracy), handwriting recognition and orientation regression (53.37% of predictions within 5° when a symbol template is provided), followed by graph assembly; they show end-to-end graph extraction qualitatively but report no end-to-end accuracy.

2.4 Relevant NLP and Validation Techniques

Hidden Markov models with Viterbi decoding [15], [16] are a classical approach to sequence labelling such as part-of-speech tagging [17]. Earley's algorithm [18] parses any context-free grammar, including ambiguous ones, in a single left-to-right pass, which makes it suitable for validating the netlist language, where a token such as "5" can be either a node or a value. Among the hand-drawn-circuit papers we reviewed, none converts English descriptions into netlists or checks the generated netlist against a formal grammar.

2.5 Comparison of Existing Approaches

Table 1**:** Comparison of existing approaches and CircuitVision

Work	Recognition method	Connectivity	Reported evaluation
Moetesum et al. 2018 [6]	Morphology + HOG + SVM	Not recovered	92% classification, 100 diagrams
Reddy & Panicker 2021 [14]	YOLOv5	Hough-transform nodes	98.2% mAP@0.5; 80% schematic rebuild (own data)
Thoma et al. 2021 [3]	Faster R-CNN (baseline)	Not addressed	52% mAP on first CGHD release
Bayer, Roy, Dengel 2023 [10]	Mask R-CNN + keypoints	Keypoint matching	Qualitative only
Bayer, van Waveren, Dengel 2024 [11]	Faster R-CNN, U-Net, CNN-MLP orientation	Graph assembly	Per-module metrics; end to end qualitative
CircuitVision (this work)	YOLOv8s + HOG/SVM orientation + EasyOCR	Box masking, connected components, crossover	Stage-wise metrics; netlist isomorphism and simulated

Work	Recognition method	Connectivity	Reported evaluation
		union-find	operating point vs reference; real CGHD results pending

2.6 Limitations of Existing Solutions

- **Evaluation stops at detection.** Few of the reviewed works check whether the reconstructed *circuit* is correct, and none checks it by simulation.
- **Results are hard to compare.** Datasets, versions and splits differ, and several works use private data.
- **Polarity is rarely handled,** although a reversed diode or source changes the circuit's behaviour.
- **Outputs are not validated** for syntax, floating nodes, missing ground or short circuits.
- **Input is image-only;** none of the reviewed systems offers a textual input sharing the same back end.

2.7 Gap Addressed by this Project

We chose to recover connectivity with classical image processing around a box detector, because CGHD's bounding boxes are enough to train it; segmentation-based graph extraction [10] needs polygon and keypoint annotations that exist for only a small part of the data. On top of this, CircuitVision recovers polarity explicitly, validates every netlist with a CFG, an Earley parser and semantic constraints, scores reconstructions against reference netlists by graph isomorphism and simulated operating point (with ground-truth and detector boxes reported separately), and adds an NLP text path that shares the same back end.

Chapter 3: Problem Definition and Proposed System

3.1 Detailed Problem Definition

Input (image path). An RGB photograph I of a hand-drawn circuit, of any resolution, possibly carrying an EXIF orientation tag, on plain or grid paper and under uneven lighting.

Input (text path). An English sentence s describing a series-parallel circuit, for example "a 9V battery driving a 1k resistor in series with (a 2k resistor in parallel with a 3k resistor)".

Output. A netlist N, that is, a set of elements $e = (ref, class, n+, n-, value)$ over integer nodes with node 0 as ground, together with a validity verdict and, if valid, the DC operating point (node voltages and source currents).

Correctness. N is correct if its circuit graph is isomorphic to the drawn circuit (elements as labelled edges between nets, with terminal order for polarised parts), its values equal the drawn values, and hence its simulated element voltages equal those of the drawn circuit. Syntactic validity alone is not sufficient.

Constraints. Two-terminal elements only; out-of-scope symbols must not corrupt the rest of the circuit; results on real data must come from a held-out drafter so that no handwriting is shared between training and test.

3.2 Proposed AI-Based Solution

We built the system as a chain of stages that can each be tested on their own, and used a learned model only where we could not write a reliable rule: finding symbols (YOLOv8s), deciding which way a diode or source points (HOG features with a linear SVM), reading handwritten values (EasyOCR) and tagging the words of a description (an HMM). Everything after detection (connectivity, netlist generation, validation and comparison) is deterministic. When a deterministic stage gives a wrong answer, the cause can be traced to a specific line of code and fixed with a regression test, which is much harder to do with a learned stage.

3.3 System Architecture

Figure 1 shows the architecture. The image path and the text path produce the same netlist representation, so they share one validator, one simulator and one evaluation harness.

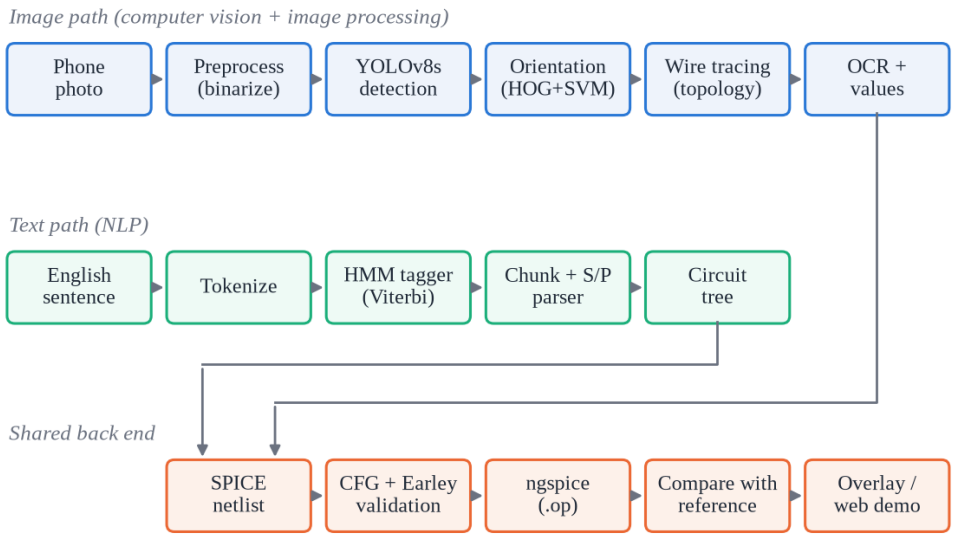


Figure 1**:** System architecture of CircuitVision: image path, text path and shared back end

3.4 Functional Requirements

Table 2**:** Functional requirements

ID	Requirement
FR1	Accept a photograph (JPEG/PNG, any size, EXIF-tagged or not) and return a SPICE netlist and an overlay image with each net coloured.
FR2	Detect resistor, capacitor, inductor, voltage source, diode, ground, junction, crossover and text symbols.
FR3	Determine the polarity of diodes and voltage sources from their orientation.
FR4	Connect wires that meet at junctions and keep crossing wires separate.
FR5	Read component values (including 4k7-style codes and decimal commas) and attach each to one component.
FR6	Choose a ground node and record the rule used in the netlist.
FR7	Validate every netlist (syntax and semantics) and report errors with line numbers.
FR8	Simulate valid netlists and report node voltages and source currents.
FR9	Accept an English series-parallel description and return a netlist through the same validator.
FR10	Provide a web demo and command-line tools, and an evaluation harness that reports every stage separately.

3.5 Non-Functional Requirements

- **Reproducibility:** fixed training configuration and seed, dataset pinned by MD5, every run's configuration and environment recorded.
- **Honesty of reporting:** every number labelled as real, synthetic/controlled, hand-written evaluation or pending; proxies named as proxies.
- **Robustness:** processing parameters scale with the size of the drawing, so the same code handles 400-pixel scans and 4,000-pixel phone photos.
- **Testability:** every stage covered by automated pytest regression tests (129 tests at the time of writing).
- **Portability:** runs on CPU for inference and tests (macOS or Linux); training uses a Colab T4 GPU.
- **Resumability:** the experiment notebook resumes after a disconnect without repeating finished stages.

3.6 Expected Outcomes

- A working image-to-netlist and text-to-netlist system with a web demo.
- A reproducible YOLOv8s baseline on CGHD with per-class test metrics.
- Stage-wise evidence of where reconstruction fails: detection, orientation, OCR or topology.
- An evidence bundle (configuration, logs, metrics, plots, tables) from which every reported number can be traced.

Chapter 4: AI Methodology

4.1 AI Techniques and Why They Suit the Problem

Table 3**:** AI techniques used and why each suits its sub-problem

Sub-problem	Technique	Why it is suitable
Finding symbols	YOLOv8s single-stage CNN detector [2], [19], fine-tuned from COCO [20]	Symbol shapes vary from drafter to drafter, so hand-made templates break; a single-stage detector is fast, and starting from COCO weights avoids training from scratch.
Symbol orientation (polarity)	HOG features [7] + linear SVM [8]	Orientation is a shape property that gradient histograms capture well; only four classes per symbol; trains in seconds and is easy to inspect.
Ink vs paper	Morphological closing + Otsu thresholding [21]	No training data needed; handles shadows and grid paper by measuring contrast relative to the local paper.
Connectivity	Connected components + union-find [22]	Whether two wires touch is a geometric question with a single right answer, so a deterministic method is easier to check and debug than a learned one.
Reading values	EasyOCR (CRAFT detection [23] + CRNN recognition [24]) [25]	Pre-trained on large text corpora; restricted to digits and unit letters to reduce misreads.
Text description	HMM tagger with Viterbi decoding [15], [16] + rule-based semantic parser	Small tag set and short sentences; an HMM trains on a synthetic corpus and handles unseen words through shape back-off.

Sub-problem	Technique	Why it is suitable
Netlist validation	CFG with Earley recogniser [18] + semantic constraints	The netlist language is ambiguous at token level; Earley keeps all readings until the grammar decides, and reports the failing line.

4.2 Dataset and its Source

4.2.1 CGHD (real data)

The real-data experiment uses CGHD [3] release 14 from Zenodo [13]: an archive of about 4.4 GB, licensed CC BY 3.0, verified by its MD5 checksum before use. Each `drafter_N` folder holds phone photographs (`images/`) and Pascal VOC XML annotations (`annotations/`). Each circuit is drawn and photographed several times, which allows a cross-photo consistency check.

Two real annotation files were inspected directly before any training, and they changed the implementation: the rotation of each symbol is stored **inside** the `<bndbox>` element at 1-degree resolution (e.g. 2, 90, 269); mirror annotations exist; and many drawings contain out-of-scope symbols such as lamps, switches and transistors.

The full release 14 archive was then audited on 2 November 2026, after its MD5 checksum was verified, together with the rotation sheets (Gate 1) and the label overlays (Gate 2). Two findings changed the implementation again. First, in 336 training images from drafters −1, 29 and 30 the XML `<size>` is a placeholder (1000×1000) while the boxes are in real image pixels; the first conversion dropped 5,321 boxes and misplaced others, and the converter now takes the size from the image itself (1 box dropped and 5 clipped; 210,782 training objects instead of 205,462; validation and test labels unchanged). Second, the test drafter contains no ground symbols and only 32 voltage sources and 40 diodes, so ground and polarity results on drafter 12 rest on few or no examples. The complete audit tables are written again by the experiment notebook; the split and class counts are **PENDING: real CGHD run**.

Table 4: Mapping from CGHD labels to the nine detector classes

CGHD labels	Class (YOLO id)	Note
resistor, resistor.adjustable, resistor.photo	resistor (0)	Drawn as a two-terminal resistor body
capacitor.unpolarized, capacitor.polarized	capacitor (1)	Capacitor polarity not modelled
inductor	inductor (2)	
voltage.dc, voltage.battery	voltage_source (3)	voltage.ac excluded (would simulate as DC)
diode, diode.light_emitting, diode.zener	diode (4)	Zener breakdown not modelled
gnd	gnd (5)	Marks node 0
junction	junction (6)	Wire dot; treated as wire
crossover	crossover (7)	Crossing wires stay separate
text	text (8)	Value label for OCR
all other labels	not a class	Ink masked during wire tracing ("other")

4.2.2 Train / validation / test separation

The split is by **drafter**, so no person's handwriting appears in both training and test data: validation is `drafter_11`, test is `drafter_12`, and all other drafters are used for training. The split was fixed

before any training. It differs from the split used by Bayer *et al.* (test drafters 23–24) [11], so the numbers are not directly comparable with theirs.

4.2.3 Synthetic data with known circuit graphs

CGHD release 14 contains bounding boxes but no ground-truth circuit graphs, so the correctness of a reconstructed circuit cannot be measured on real photos. To measure topology exactly, a generator (`synth.py`) draws five circuits with known graphs (a voltage divider, a parallel RC with junction dots, two independent loops crossing twice, a series RLC and a Wheatstone-style bridge) under eleven photo conditions (Figure 2). It can also write a synthetic tree in the CGHD format, with reference netlists, so that the full evaluation harness runs end to end. **Results on these data are labelled SYNTHETIC and say nothing about real photos.**

For the real test drafter, reference netlists were written for the 12 drawings that contain in-scope symbols (circuits C133–C138, two drawings each; C139–C144 are logic-gate diagrams with nothing to reconstruct). They were drafted from the photos without running the pipeline and checked against the photos by a team member. Out-of-scope parts (switches, AC sources) are left out of these references with their terminals kept apart, as the pipeline masks them, so only circuit C137 is complete enough to simulate.

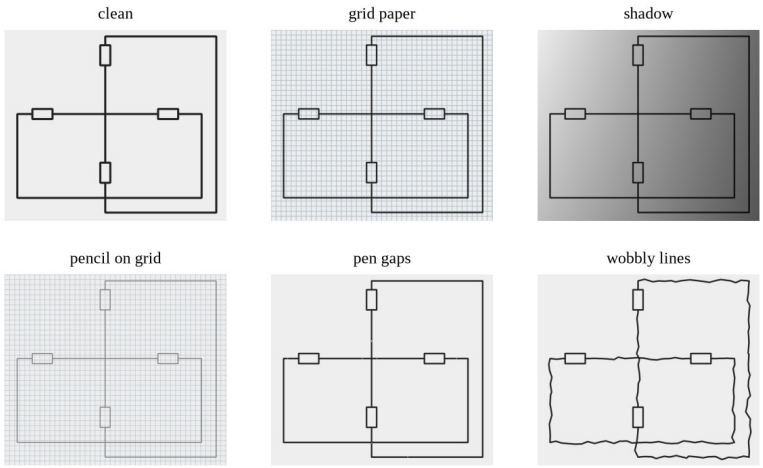


Figure 2: Six of the eleven synthetic photo conditions applied to the crossover circuit

4.2.4 Text data

The HMM tagger is trained on 3,000 sentences generated from templates with lexical variation (`nlp/corpus.py`). Evaluation uses three **hand-written** sets that the training corpus never sees: a 30-sentence set frozen before the NLP improvements (later used for error analysis, so its post-change score is optimistic), a fresh 20-sentence set written after development and evaluated exactly once, and a second 20-sentence set written by a teammate who had not seen the parser or the earlier sets, kept exactly as typed (final full stops included) and also evaluated exactly once.

4.3 Data Preprocessing

4.3.1 Annotation conversion

Each VOC object is mapped through the explicit class table (Table 4); boxes are clipped to the image and zero-area boxes are dropped, with every change counted in `conversion_stats.json`. YOLO labels are written as `class cx cy w h`, normalised by the annotated image size.

4.3.2 EXIF orientation

Phone photos often carry an EXIF orientation tag. Each image is decoded in the pixel frame its annotation refers to, chosen by matching the annotated size against the raw and EXIF-rotated frames. Images with an EXIF rotation are re-encoded without the tag, so that the trainer and every tool see exactly the pixels the boxes refer to, and these images are always included in the visual label check.

4.3.3 Binarisation

Wire tracing needs a clean ink mask. Our first version used OpenCV's adaptive threshold; on grid paper it kept the grid lines, which then joined every wire into a single net. We replaced it with a contrast measure relative to the local paper brightness (Figure 3):

1. Convert to grey and apply a 5×5 Gaussian blur.
2. Estimate the paper brightness P with a grey-level morphological closing whose square window is 1/15 of the image's shorter side (at least 15 px). The closing removes every dark feature thinner than the window (pen strokes, grid lines, filled junction dots) and follows shadows.
3. Compute relative contrast $c = 255 \cdot (P - I) / P$, so a stroke in shadow scores like one in daylight.
4. Threshold c with Otsu's method [21] on the candidate pixels, with a floor of 12; apply a 2×2 opening and remove connected components smaller than 30 px.

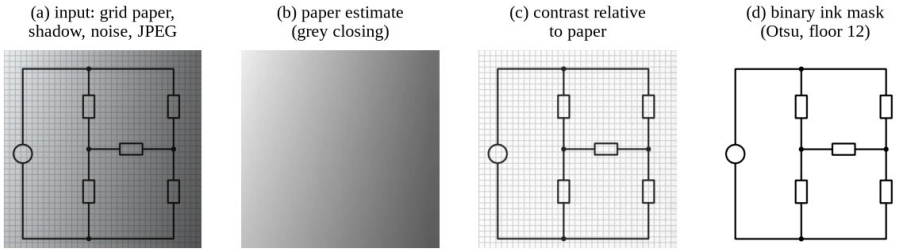


Figure 3**:** Binarisation stages on a synthetic photo with grid paper, a shadow, noise and JPEG compression

4.4 Feature Extraction and Representation

- **Detector:** features are learned by the YOLOv8s backbone; input images are resized to 1024 px.
- **Orientation:** each crop is padded to a square, resized to 64×64 and described by a HOG vector (16×16 blocks, 8×8 cells, 9 orientation bins). Every training crop is also rotated by 90°, 180° and 270° with its label shifted accordingly, so one labelled crop yields four samples (Figure 4).
- **Text:** each word is lower-cased; words seen fewer than twice in training, and a random 5% of training tokens (word dropout), are replaced by a case-aware *shape* class (e.g. number-like), so that unseen values and part numbers still receive sensible emission probabilities.
- **Circuit:** the shared representation is the netlist; for evaluation it is converted to a labelled multigraph (elements as edges between nets) for isomorphism testing.

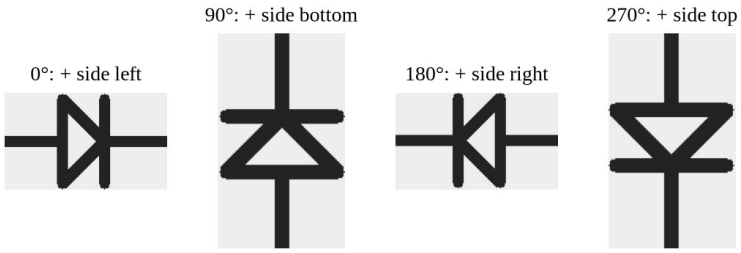


Figure 4**:** Rotation augmentation of a synthetic diode crop; the + side follows from the label under the convention fixed at Gate 1 (anode or + on the left at 0°, counter-clockwise)

4.5 Model Selection and Training

4.5.1 Component detector

We used the small YOLOv8 model (YOLOv8s), fine-tuned from COCO-pretrained weights, as a compromise between accuracy and what a free Colab T4 GPU can train in one session (we estimate 3–5 hours for the full run). The configuration (Table 5) was fixed in code before any training, so it cannot drift between runs; each run records it together with the library and GPU versions.

Table 5**:** Fixed baseline training configuration (baseline_v1)

Setting	Value
Model	YOLOv8s, COCO-pretrained yolov8s.pt
Image size / batch	1024 px / 16
Epochs	100, early-stopping patience 30
Optimiser	SGD, lr0 0.01, lrf 0.01, momentum 0.937, weight decay 5×10^{-4}, 3 warm-up epochs
Augmentation	flips 0.5/0.5, rotation ±5°, scale 0.5, translate 0.1, HSV s 0.3 / v 0.4, mosaic 1.0 (off for the last 10 epochs)
Seed	0, deterministic
Classes	9 (see Table 4)
Split	train: all drafters except 11 and 12; validation: drafter 11; test: drafter 12

4.5.2 Orientation classifier

One linear SVM per polarised class (diode, voltage source) is trained on HOG vectors of CGHD crops, with each rotation rounded to the nearest 90°. Mirrored symbols are excluded from training and scoring because the effect of a mirror on polarity depends on undocumented symbol templates. Internally, rotation is counter-clockwise and, at rotation 0, the positive terminal lies on a configured side.

How CGHD's rotation values map to this convention was decided at **Gate 1** on 2 November 2026 from sheets of real crops: for diodes, LEDs, Zener diodes and batteries the anode or + is on the left at 0° and the symbol turns counter-clockwise, which agrees with the dataset's own port definitions for diodes. One label is an exception: all 182 rotated `voltage.dc` crops are labelled 0° with + on top, so 270° is added to that label when annotations are read (the raw value stays visible to the audit). A second team member reviewed both gate decisions independently on 4 November 2026.

A preliminary local check on test drafter 12 gave 0.938 orientation accuracy on the 32 voltage sources (0.875 before the `voltage.dc` correction) and 0.950 on the 40 diodes; because these test crops were seen while choosing the correction, they are not reported as a result.

4.5.3 HMM tagger

A first-order HMM over 11 tags (SRC, RES, CAP, IND, DIO, VAL, UNIT, SER, PAR, SEP, O) is estimated by counting with add-k smoothing (k = 0.1) and decoded with the Viterbi algorithm [16] in log space. With tags *t* and words *w*, Viterbi finds argmax over $t_1 \ldots t_n$ of $\log \pi(t_1) + \Sigma \log A(t_{i-1}, t_i) + \Sigma \log B(t_i, w_i)$.

4.5.4 OCR

EasyOCR [25] is used pre-trained, without fine-tuning, restricted to the characters `0-9 . , k K m M u µ n p G R r F f H h V v Ω`. Labels taller than 1.5× their width are rotated upright before recognition.

4.6 System Workflow

Figure 5 shows the protocol of the real-data experiment, implemented as a single Colab notebook. Two human gates are `assert` cells that stop execution until approval and notes are recorded: Gate 1 fixes the rotation convention after the dataset audit (the decided values are prefilled, but a person must still approve), and Gate 2 confirms visually, on 24 overlays covering every split and class and the EXIF-rotated photos, that the converted labels sit on their symbols. Every step is logged, stops on failure and writes its outputs to Google Drive; after a disconnect the notebook skips finished stages and resumes training from the last checkpoint.

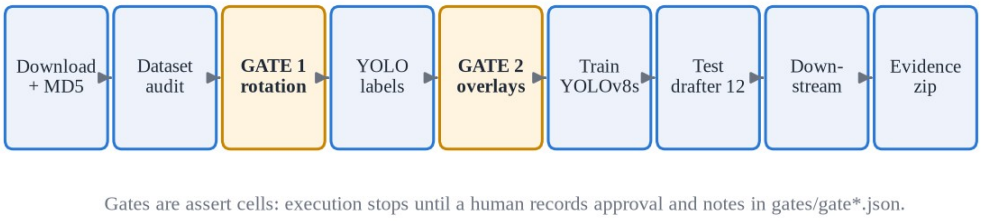


Figure 5**:** Real-data experiment protocol with two human verification gates

4.7 Tools, Libraries and Technologies

Table 6**:** Tools, libraries and technologies

Tool	Version	Used for
Python	3.11 (≥ 3.10 supported)	All code
Ultralytics YOLO [19] / PyTorch	8.4.184 / 2.x	Detector training and inference
OpenCV (headless, pinned) [26]	4.10.0.84	Image processing, HOG, SVM
NumPy, Pillow	—	Arrays, image headers
EasyOCR	1.7.2	Reading values
ngspice [27]	42 (Linux), 47 (macOS)	Circuit simulation
Gradio [28]	≥ 5	Web demo
pytest	—	129 regression tests
Google Colab (T4 GPU) + Drive	—	Training and evaluation runs
Git / GitHub	—	Version control, pull requests

Chapter 5: Implementation

5.1 Software and Hardware Requirements

Table 7**:** Software and hardware requirements

Item	Development / inference	Training
Operating system	macOS (Apple Silicon) or Linux	Google Colab (Linux)
Processor / GPU	Any modern CPU; Apple MPS optional	NVIDIA T4 GPU (16 GB)
Memory	8 GB RAM	Colab default
Storage	≈ 2 GB for environment	≈ 10 GB local (dataset + converted copy); Drive for results
Software	Python ≥ 3.10, package with extras detect/app/dev, ngspice	Same, installed by the notebook

Training on a laptop is not practical at the fixed settings: on a 16 GB Apple M5, one batch took about 132 s, which is about 29 days for 100 epochs. Training therefore runs on a Colab T4 GPU, and inference and all tests run on an ordinary CPU.

5.2 Module-wise Description

Table 8**:** Modules of the circuitvision package

Module	Responsibility
preprocess.py	Image loading and paper-relative binarisation
cghd_to_yolo.py	VOC parsing (rotation inside bndbox, mirror), EXIF-aware loading, drafter-wise YOLO export
audit.py, visualize.py	Full dataset audit; rotation sheets (Gate 1); stratified label overlays (Gate 2)
detect.py, train.py, detector_eval.py	YOLO inference; fixed-configuration training with automatic resume; test metrics and failure categories
orientation.py	Rotation convention, HOG + SVM orientation model
topology.py	Wire tracing: box masking, wire labelling, terminal rings, crossovers
ocr.py, netlist.py	EasyOCR; value parsing; label-to-element assignment; ground rule; SPICE text
grammar.py	CFG, Earley recogniser, semantic checks
simulate.py, compare.py	ngspice operating point; isomorphism and operating-point comparison with a reference
pipeline.py, app.py	End-to-end command line and Gradio web demo
evaluate.py, report_tables.py, synth.py	Stage-wise evaluation; tables generated only from result files; synthetic data
nlp/ (lexicon, hmm, corpus, parser, tree, text2netlist, evaluate)	Text-to-netlist path and its evaluation

5.3 Important Algorithms

5.3.1 Wire tracing

Wire tracing turns the ink mask and the symbol boxes into nets (Figure 6). The idea is simple: erase every symbol from the mask, so that only wires are left, and treat each remaining connected piece of ink as one net. The usual alternative is to skeletonise the wires and follow line segments; erasing boxes needs no line-following rules and is easy to test, so we started with it. The price is that two wires drawn

touching each other are merged, and a crossing is kept apart only if the detector finds the crossover symbol. All sizes scale with *s*, the median short side of the element boxes divided by 24, so a 4,000-px photo behaves like a 400-px drawing (Section 6.9 describes what went wrong before this was added).

```
Algorithm 1: extract_nets(mask, boxes)
  s <- max(1, median short side of element boxes / 24)
  grow every box by 3s; erase element, ground, crossover, text and
      out-of-scope boxes from the mask           # junction dots stay ink
  dilate the remaining ink by max(3, 3s)         # bridge pen gaps
  label 8-connected components -> candidate wires
  drop components smaller than 40 s^2, and components that never leave
      the neighbourhood of a single box          # outline slivers
  for each crossover box:
      union(labels on left side, labels on right side)
      union(labels on top side,  labels on bottom side)
  for each element box:
      read wire labels in a ring of width 6s on each side
      order sides: + side first if polarity known, else
          left,right (wide box) or top,bottom (tall box)
      terminals <- distinct union-find roots in that order
  return terminals per element
```

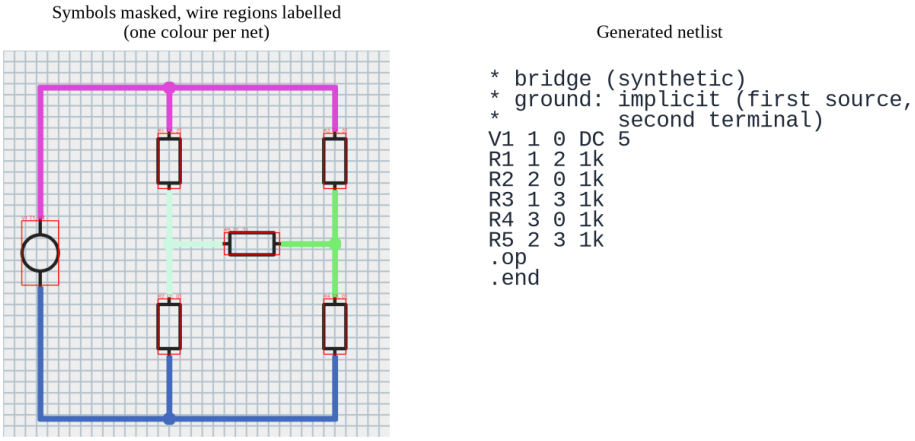


Figure 6**:** Wire tracing on a synthetic bridge drawn on grid paper: nets coloured, and the generated netlist

5.3.2 Value attachment

```
Algorithm 2: attach_values(elements, text_boxes)
  labels <- [(box, value, unit_class) for OCR text that parses as a value]
  pairs  <- []
  for each element e, label l:
      if l.unit_class is set and differs from e.class: skip  # no 10uF on a resistor
      d <- distance between box centres
      if d <= max(150, 1.5 x longer side of e): pairs.append((d, e, l))
  for (d, e, l) in pairs sorted by d:                    # greedy, closest first
      if e and l both unused: e.value <- l.value; mark both used
```

Assigning each element its nearest label fails when two resistors share one nearby label: both take it. Taking pairs in order of distance and using each label once avoids that, and the unit check stops a capacitor value from landing on a resistor that happens to be closer. Value parsing accepts the forms seen on drawings: `10k`, `4.7uF`, `4,7µF` (decimal comma), `220Ω`, RKM codes (`4k7` → 4.7k, `2R2` → 2.2, `1M5` → 1.5 Meg) and trailing-R ohm values (`330R` → 330 Ω, a resistor). Names such as `R1` are not values and are ignored.

5.3.3 Earley recogniser for the netlist grammar

The grammar covers the generated SPICE subset: a title comment, element lines `R|C|L name node node value`, `V name node node [DC] value`, `D name node node model`, `.model`, `.op` and `.end`. Terminals are regular-expression predicates rather than fixed token types, so a token such as `1` can be a node or a value.

```
Algorithm 3: earley(tokens)
  chart[0] <- {(S -> . alpha, 0) for each rule of S}
  for i in 0..n:
      for each item (A -> beta . X gamma, j) in chart[i] (agenda):
          if X is a non-terminal: add (X -> . delta, i) to chart[i]      # predict
          elif X matches tokens[i]: add (A -> beta X . gamma, j) to chart[i+1]
      for each completed item (B -> delta ., j) in chart[i]:      # complete
          advance each (A -> beta . B gamma, k) in chart[j] into chart[i]
      if chart[i+1] is empty: return (reject, i)   # error position -> line number
  accept iff (S -> alpha ., 0) in chart[n]
```

After a successful parse, semantic checks reject: a missing ground node, floating nodes (fewer than two connections), shorted elements, duplicate references and diodes that use an undefined model.

5.3.4 Viterbi decoding

```
Algorithm 4: viterbi(words)
  score[0, t] <- log pi[t] + log B[t, w0]
  for i in 1..n-1:
      for each tag t:
          back[i, t]  <- argmax_u score[i-1, u] + log A[u, t]
          score[i, t] <- max_u (score[i-1, u] + log A[u, t]) + log B[t, wi]
  follow back-pointers from argmax_t score[n-1, t]
```

5.4 Implementation Details

- **Ground.** Nets touching a ground symbol become node 0. Many drawings have no ground symbol; then the second (negative, if polarity is known) terminal of the first source becomes node 0, or failing that the most connected net. The rule used is written into every netlist header. Because comparison uses element voltages, the choice of reference node does not affect correctness.
- **Defaults.** Without a readable value, components get placeholder defaults (R 1k, C 1u, L 1m, V DC 5); diodes use a generic `.model DMOD D`. An element without exactly two terminals is not emitted but flagged as a `* WARNING` comment, so a broken reconstruction is visible rather than silently wrong.
- **Text parser.** Each component noun takes its nearest unclaimed value; an explicit unit decides the class; count words ("two 1k resistors") repeat elements; parentheses group explicitly; a comma followed by an operator closes the preceding clause ("A and B in parallel, in series with C" → S(P(A,B),C)); otherwise operators attach to the right, as in prepositional-phrase attachment. The source is placed across the whole load.
- **Evaluation.** `evaluate.py` runs every downstream stage twice, with ground-truth boxes (an oracle detector) and with detector boxes, on all images and on the subset without out-of-scope symbols, and writes CSV files, netlists and a JSON summary. `report_tables.py` builds the results tables only from these files and prints NOT AVAILABLE when an artefact is missing.

- **Testing and reproducibility.** 129 pytest tests across 14 files cover every stage, including one real CGHD annotation file as a regression fixture. The experiment notebook pins package versions, records the commit, the dataset checksum, the GPU and the full package list, and bundles all evidence into a ZIP file.

5.5 Screenshots of the Running System

Figure 7 shows the Gradio web demo processing a synthetic drawing with a detector trained on synthetic data (the real-data model is pending). The overlay colours each net; the netlist reads the handwritten-style values `4k7`, `9V` and `220`; validation passes; and ngspice gives v(2) = 0.4024 V, which matches 9 × 220 / (4700 + 220). Figure 8 and Figure 9 show the command-line tools and the test suite.

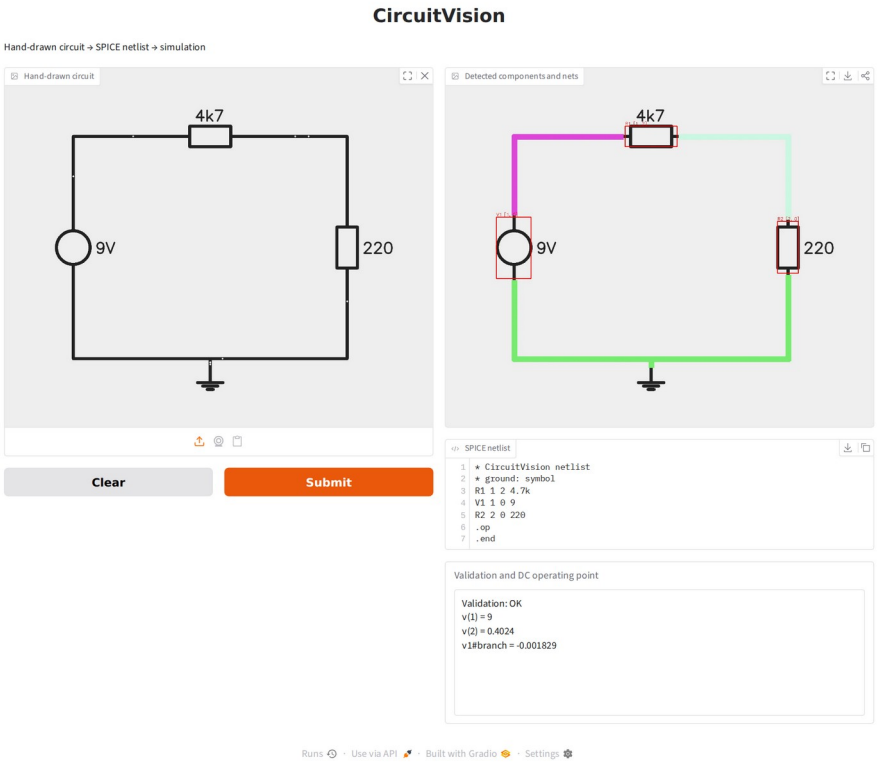


Figure 7**:** Gradio web demo: input, detected components and nets, SPICE netlist, validation and DC operating point (synthetic input and model)

```
Terminal: image path with ground-truth boxes, validation and ngspice

$ python -m circuitvision.pipeline examples/divider.png --xml examples/divider.xml
    --simulate
* divider
* ground: symbol
V1 1 0 DC 5
R1 1 2 10k
R2 2 0 1k
.op
.end

validation: OK
  v(1)         5
  v(2)         0.4545
  v1#branch    -0.0004545
```

Figure 8**:** Command-line image path on the example drawing with ground-truth boxes, validation and simulation

```
Terminal: text path (tags, netlist, validation) and test suite

$ python -m circuitvision.nlp.text2netlist "a 12V battery driving a 1k resistor in series
    with (a 2k resistor in parallel with a 10uF capacitor)"
a/O 12V/VAL battery/SRC driving/O a/O 1k/VAL resistor/RES in/O series/SER with/O (/O a/O
    2k/VAL resistor/RES in/O parallel/PAR with/O a/O 10uF/VAL capacitor/CAP )/O

* a 12V battery driving a 1k resistor in series with (a 2k res
V1 1 0 DC 12
R1 1 2 1k
R2 2 0 2k
C1 2 0 10u
.op
.end

validation: OK

$ pytest -q
107 passed in 15.27s
```

Figure 9**:** Command-line text path (tags, netlist, validation) and the test suite

5.6 Sample Inputs and Outputs

(a) input drawing (examples/divider.png) (b) output overlay: nets coloured, refs and nodes

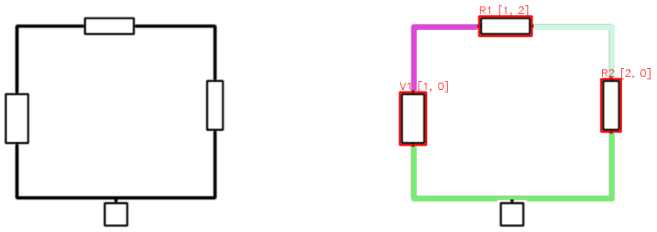


Figure 10**:** Example input drawing and the output overlay produced by the pipeline

Table 9**:** Sample text inputs and generated netlists (actual outputs)

Input sentence	Generated netlist (elements)	Validation
a 9V battery driving a 1k resistor in series with (a 2k resistor in parallel with a 3k resistor)	V1 1 0 DC 9; R1 1 2 1k; R2 2 0 2k; R3 2 0 3k	OK
a 12V battery driving a 1k resistor in series with (a 2k resistor in parallel with a 10uF capacitor)	V1 1 0 DC 12; R1 1 2 1k; R2 2 0 2k; C1 2 0 10u	OK

Table 10**:** Validator outputs on deliberately faulty netlists (actual outputs)

Faulty input	Reported error
R2 line with a missing value (`R2 1 0`)	syntax error at line 3
Ground reached by only one element	node 0 is floating (1 connection)
Diode referring to model DX that is not defined	D1 uses undefined model DX

Chapter 6: Results and Discussion

6.1 Evaluation Design

Every result in this chapter carries one of four labels: **REAL CGHD** (measured on the real dataset), **SYNTHETIC** (data generated by our code, where the right answer is known), **HAND-WRITTEN** (sentences we wrote for NLP evaluation) or **PENDING** (not yet measured). Three principles run through the evaluation:

- **Isolate stages.** Every downstream stage is evaluated with ground-truth boxes and with detector boxes, separating detection errors from reconstruction errors.

- **Validate the circuit, not the syntax.** Netlists are compared with references by graph isomorphism (*topology match*), isomorphism with equal values (*exact match*) and equal simulated element voltages within 0.1% (*operating-point match*).
- **Name proxies as proxies.** On real CGHD, where no reference graphs exist, topology is reported only through proxies: the share of elements with exactly two terminals, and the consistency of the reconstructed graph across photos of the same drawing.

6.2 Evaluation Metrics

Table 11**:** Evaluation metrics by stage

Stage	Metric	Definition / caveat
Detection	Precision, recall, mAP@0.5, mAP@0.5:0.95, per-class AP, confusion matrix	Ultralytics validation on test drafter 12; error categories at IoU 0.5, confidence 0.25
Orientation	Accuracy	Against CGHD rotation binned to 90°, mirrored symbols excluded; valid only under the Gate 1 convention
OCR	Exact match; character accuracy (1 − CER)	On ground-truth text boxes; all labels and value-like labels
Topology	Topology / exact / operating-point match	Only where a reference netlist exists: synthetic data, and 12 drafter-12 drawings (operating point only for the one complete circuit)
Topology (real)	2-terminal share; cross-photo consistency	Proxies: consistent is not the same as correct
Netlist	Valid share; simulates share	A valid netlist is not necessarily the drawn circuit
Text path	Token accuracy, per-tag P/R/F1, macro F1, exact circuit match, component P/R/F1	Tagging scored on gold tokens; circuit scored from the raw sentence

6.3 Real CGHD Results (baseline_v1)

The real-data experiment is fully implemented and has been tested end to end on synthetic data in the CGHD format, but it had not been run on the real dataset at the time of writing. **No real-photo result is claimed in this report.** The dataset checks, Gate 1 and Gate 2 are complete and the experiment is ready to run (Section 4.2.1). The values in Table 12 will be filled only from the experiment's evidence files (`test/metrics.json`, `downstream/summary.json` and the generated `tables.md`). Real topology will be scored against the 12 reference netlists of drafter 12 (Section 4.2.3); only circuit C137 is complete enough for an operating-point comparison.

Table 12**:** Real CGHD results on test drafter 12 (REAL CGHD)

Measure	Value	Source file
Dataset audit: drafters, images, objects per split, class counts	**PENDING: real CGHD run**	audit/audit.md
Detection: mAP@0.5 / mAP@0.5:0.95	**PENDING: real CGHD run**	test/metrics.json
Detection: precision / recall	**PENDING: real CGHD run**	test/metrics.json
Per-class AP and confusion matrix	**PENDING: real CGHD run**	test/metrics.json, confusion_matrix.csv
Orientation accuracy (after Gate 1)	**PENDING: real CGHD run**	orientation.json
OCR exact / character accuracy	**PENDING: real CGHD run**	downstream/ summary.json
Downstream, GT boxes: valid / simulates / 2-terminal	**PENDING: real CGHD run**	downstream/

Measure	Value	Source file
share / consistency		summary.json
Downstream, detector boxes: same measures + agreement with GT-box netlist	**PENDING: real CGHD run**	downstream/ summary.json

6.4 Wire Tracing on Synthetic Photos (SYNTHETIC)

With ground-truth boxes, the tracer recovered the exact terminal-to-net partition in **99.4%** of 165 trials (5 circuits × 11 conditions × 3 seeds) (Figure 11). The last five conditions were added after the first six passed, and the method was not tuned to them. The only failure occurred under a simulated detector box error of 15%: on one of three seeds of the crossover circuit, a shifted crossover box no longer covered the crossing, so two crossing wires were joined.

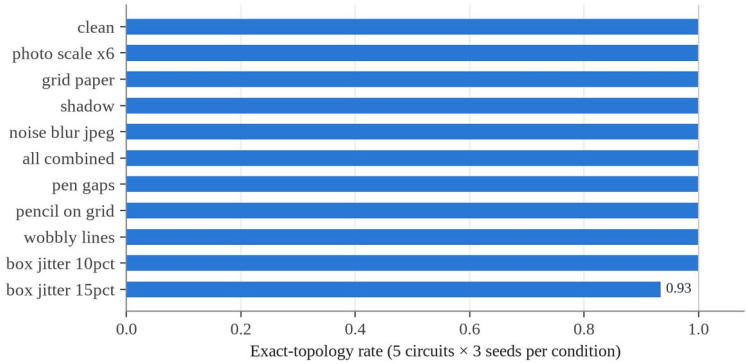


Figure 11**:** Exact-topology rate per photo condition (SYNTHETIC, ground-truth boxes)

6.5 End-to-End Reconstruction against Reference Netlists (SYNTHETIC)

The full evaluation harness was run on CGHD-format synthetic data with reference netlists (Table 13). With ground-truth boxes, every circuit matched its reference exactly, including the simulated operating point. To exercise detector mode, a YOLOv8s model was trained for 40 CPU epochs on synthetic drafters (not the baseline configuration) and run on synthetic test drafter 12.

Table 13**:** End-to-end results on CGHD-format synthetic data (SYNTHETIC)

Mode	Images	Valid	Simulates	Topology match	Exact match	Op-point match
GT boxes (drafters 10–12)	18	18/18	18/18	18/18	18/18	18/18
GT boxes (drafter 12)	6	6/6	6/6	6/6	6/6	6/6
Detector boxes (drafter 12)	6	6/6	5/6	5/6	4/6	4/6

Both detector-mode failures are easy to explain. In one bridge circuit the graph was right, but the middle resistor came out as 1k instead of the drawn 10k, so the topology matched while the exact and operating-point comparisons failed. In the crossover circuit the detector missed both crossover symbols (Figure 12), so the two independent loops were merged into one net and the netlist did not produce an operating point. This is a single synthetic example, but it shows the failure we most expect on real photos: a missed crossover symbol breaks the topology even when every component is found. It also shows that the evaluation separates a detection error from a reconstruction error, as intended.

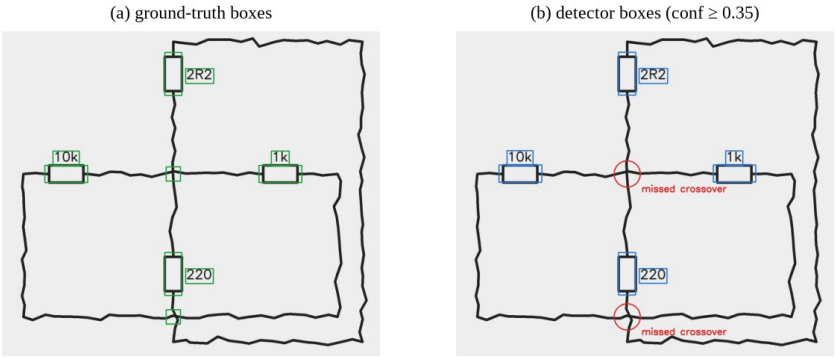


Figure 12**:** Detector-mode failure on a synthetic crossover circuit: both crossover symbols are missed, merging two independent loops

6.6 OCR and Orientation (SYNTHETIC)

On synthetic value labels, EasyOCR reached 0.914 exact match and 0.950 character accuracy over 70 labels (drafters 10–12), and 0.958 / 0.990 over the 24 labels of drafter 12. The HOG + SVM orientation classifier was correct on 5 of 5 polarised symbols of held-out synthetic drafter 12 (drafters 10 and 11 were in its training data and are not counted). Real OCR and orientation accuracy are **PENDING: real CGHD run**.

6.7 Text-to-Netlist Results (HAND-WRITTEN)

Table 14**:** Text-to-netlist results on hand-written evaluation sets

Set	Sent.	Token acc.	Macro F1	Exact circuit	Component F1	Valid netlist
30-sentence set, before NLP changes	30	0.986	0.975	0.800	0.965	1.000
30-sentence set, after changes (optimistic)	30	0.995	0.980	1.000	1.000	1.000
Fresh 20-sentence set, evaluated once	20	1.000	1.000	0.900	0.972	1.000
Teammate 20-sentence set, evaluated once	20	0.994	0.990	0.750	0.986	1.000

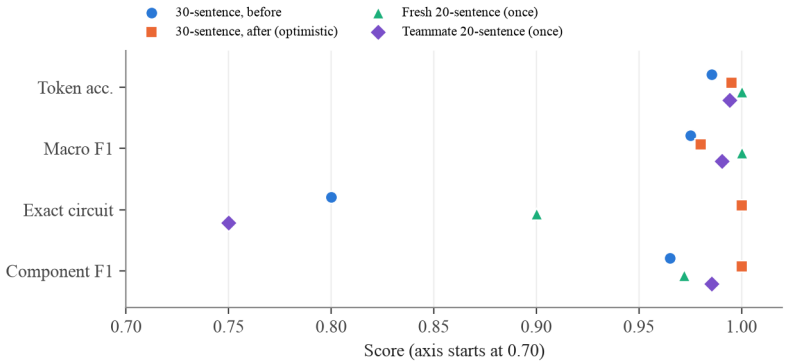


Figure 13**:** Text-to-netlist scores on the four evaluation runs (HAND-WRITTEN)

The 30-sentence set was frozen before any NLP change, but development then targeted the error categories its baseline exposed, so its after-score is optimistic. The fresh set, written after development and evaluated once, gives 90% exact circuit match, with every generated netlist valid.

The teammate's set is the most independent test, because its author had not seen the parser: 75% exact circuit match, still with every netlist valid and component F1 0.986. All five failures come from how a

sentence arranges the parts, not from unknown words or values (the evaluator files them as two tagging, one structure and two component errors): "across both of them" meaning parallel, a physical order given with "through", a parallel group placed before "connected in series with", and two parts attached "across" a single earlier part. The two independent sets together suggest that the parser handles vocabulary and values well but only the common sentence structures.

On the synthetic corpus, the tagger reaches 1.000 token accuracy on held-out sentences from the training templates, but 0.897 on phrasings never used in training (exact circuit 0.980), showing that the semantic constraints recover much of what the tagger misses.

Table 15**:** Failures on the fresh 20-sentence set

Sentence	Expected	Produced	Cause
a 9 volt battery with a 330R resistor and an LED in series	S(resistor 330, diode)	S(resistor default, diode)	Trailing-R ohm notation "330R" not parsed as a value (fixed later)
a pair of 10k resistors in parallel	P(resistor 10k, resistor 10k)	single resistor 10k	Count phrase "a pair of" not in the count lexicon (fixed later)

Both failures were lexical gaps, and both were fixed afterwards. The "330R" fix also applies to the image path, where a handwritten "330R" label had been ignored. Re-running every set after these fixes gives 1.000 exact circuit match on the fresh set, but that figure is optimistic because the fixes target exactly its two failures, so the once-evaluated 0.900 remains the reported result. The other sets are unchanged (30-sentence set 1.000; the teammate's set 0.750 with identical output). The re-run also corrected the error categories of the teammate's set: a final full stop, which the tokenizer drops by design, had made every failure count as a tokenisation error; the five failures are now classified as two tagging, one structure and two component errors, with the scores unchanged.

6.8 Comparison with Baselines

- **Internal baselines.** Before the scale-adaptive tracer, the tracer tuned on small drawings recovered 0 of 30 synthetic circuits even on clean images; after the changes it recovers 99.4% across eleven conditions. The text path improved from 0.800 to 1.000 exact match on its development set (optimistic), and scores 0.900 on fresh data.
- **Oracle vs detector.** On synthetic test data, operating-point match falls from 6/6 with ground-truth boxes to 4/6 with detector boxes, which isolates the cost of detection errors.
- **Published work.** Published CGHD results (e.g. 52% mAP on the first release [3]; 18% mAP on drafters 21–22 of a later release [11]) use different releases, splits and detectors, and a YOLOv5 result of 98.2% mAP@0.5 [14] uses a different dataset. Our real detection results are **PENDING: real CGHD run**, and even then they will not be directly comparable because the test drafter differs.

6.9 Defects Found during Development

Most of the serious bugs were not found by unit tests. They appeared when we ran the code on data in the real format or compared its output with reference netlists (Table 16). Three examples show how this

went. Our parser first read each symbol's rotation from the `<object>` element; a real annotation file showed that it is nested inside `<bndbox>`, so every rotation would have been read as missing and the orientation classifier would have had no labels. Our first wire tracer, tuned on small test drawings, got 0 of 30 synthetic circuits right even on clean images, because at phone-photo resolution pieces of each symbol's outline stuck out of its box and were counted as extra wires. And on the first Colab run the orientation stage crashed with `cv2.HOGDescriptor` missing, because several OpenCV packages had overwritten the same folder. Each code fix is covered by a regression test; the two installation fixes are checked by `scripts/check_opencv.py` before every GPU step.

Table 16**:** Defects found during development and their fixes

Failure	Found by	Cause	Fix
Every CGHD rotation read as missing	A real annotation file	Rotation is nested inside bndbox	Parser reads both levels
0/30 synthetic circuits correct, even clean	Topology benchmark	Outline slivers read as terminals; pixel sizes tuned for small drawings	Sliver rejection; sizes scale with symbol size
Grid paper merges all nets	Topology benchmark	Adaptive threshold keeps grid lines	Paper-relative contrast + Otsu
Wires split at junction dots	End-to-end run vs references	Closing window smaller than a filled dot	Window = 1/15 of the image side
"No ground node" on drawings without a ground symbol	End-to-end run	Ground assumed to be drawn	Implicit ground rule, recorded in the netlist
Out-of-scope symbols short wires	Real annotations	Lamps and switches not masked	Masked as class "other"
Colab run failed: cv2.HOGDescriptor missing	First Colab run	Several OpenCV builds overwrite one folder	Pin exactly one OpenCV build
voltage.dc polarity reversed	Gate 1 rotation sheets	This label uses + on top at 0°, unlike every other polarised symbol	Add 270° when reading voltage.dc
5,321 training boxes dropped, others misplaced	Gate 2 label overlays	Placeholder XML size (1000×1000) in drafters −1, 29, 30	Take the size from the image
Base install broke orientation	Teammate's reproduction	OpenCV 5 has no cv2.HOGDescriptor	Cap OpenCV below 5
Handwritten "330R" values ignored	Fresh NLP test set	Trailing-R ohm notation not parsed (text and image path)	Parse it as a resistor value

6.10 Limitations and Observed Errors

- **No real-photo accuracy yet.** Every accuracy figure here is synthetic or from hand-written sentences; real CGHD results are pending.
- **Real topology rests on 12 references.** CGHD has no circuit graphs; the 12 hand-checked drafter-12 netlists cover the in-scope drawings, and only C137 can be simulated. Elsewhere, only proxies are available.
- **Synthetic drawings are cleaner than real ones:** five circuit types, simple symbols, machine-drawn text.
- **Crossovers are fragile.** A missed or badly placed crossover box merges two nets (Section 6.5).

- **Polarity rests on few test examples.** The rotation convention is fixed at Gate 1, but test drafter 12 has only 32 voltage sources and 40 diodes, and no ground symbols; mirrored symbols are excluded.
- **Limited scope:** two-terminal elements, DC operating point, series-parallel text only.
- **Small evaluation sets:** 20–30 hand-written sentences, 6–18 synthetic test images.

6.11 Ethical and Privacy Considerations

- **Licence.** CGHD is used under CC BY 3.0, credited, and not redistributed.
- **Privacy.** Processing is local; photos leave the machine only if Gradio's public share link is enabled. Handwriting is weakly personal, so photos should not be shared without the drafter's consent.
- **Over-trust.** A valid, simulating netlist can still be the wrong circuit; a person should review it before any safety-relevant use.
- **Honest reporting.** Real, synthetic, hand-written and pending results are labelled, and no missing result has been estimated.

Chapter 7: Conclusion and Future Scope

7.1 Summary of the Work

We built a system that turns a photo of a hand-drawn circuit, or a sentence describing one, into a SPICE netlist, checks it with a context-free grammar and an Earley recogniser, and simulates it in ngspice. The learned parts (YOLOv8s detection, HOG + SVM orientation, EasyOCR and the HMM tagger) are kept to the places where rules do not work; the rest (binarisation, wire tracing, netlist generation and validation) is deterministic and covered by regression tests. Much of the effort went into evaluation: running every stage with ground-truth and detector boxes, scoring netlists against reference circuits rather than only checking that they parse, and keeping synthetic, hand-written and real results apart.

7.2 Objectives Achieved

Table 17**:** Status of the objectives

Objective	Status
1. YOLOv8s detector on real CGHD	Implemented, configured and tested on synthetic data; real training and test PENDING
2. Polarity from orientation	Implemented; Gate 1 convention decided; 5/5 on synthetic test data; real result PENDING
3. Robust wire tracing	Achieved on synthetic data: 99.4% exact topology over 11 conditions
4. Value reading and attachment	Implemented; 0.914–0.958 exact on synthetic labels; real result PENDING
5. Netlist, validation, simulation	Achieved: valid netlists and correct operating points on all synthetic GT-box cases
6. Text-to-netlist	Achieved: 0.90 and 0.75 exact circuit match on two independent hand-written sets
7. Stage-wise, labelled evaluation	Achieved: harness, gates, evidence bundle and tables generated from artefacts

7.3 Key Findings

- Measuring ink contrast relative to the local paper, instead of thresholding brightness, is what made grid paper and shadows work.
- Every size in the wire tracer must scale with the drawing; parameters tuned on small drawings failed completely on phone-photo resolutions.
- Testing against the real annotation format and against reference netlists found defects that unit tests did not, including one that would have silently invalidated all orientation results.
- Crossover detection is critical: missing one merges two nets, even when every other symbol is detected correctly.
- A netlist that parses is not necessarily the right circuit; graph isomorphism and simulated operating points are the meaningful checks.

7.4 Current Limitations

The main limitation is that the real CGHD experiment had not been run at the time of writing, so the system's accuracy on real photos is unknown. Beyond that: real topology can be scored exactly only on the 12 drafter-12 drawings with reference netlists; scope is limited to two-terminal elements and DC analysis; and the text path handles only series-parallel descriptions.

7.5 Future Improvements

1. Run baseline_v1 on a Colab T4 (Gates 1 and 2 are already decided) and fill Section 6.3 from the evidence bundle.
2. Extend the reference netlists beyond the 12 drafter-12 drawings, so real topology is measured on more than a handful of circuits.
3. Add a second run with the split used in the literature (test drafters 23–24) for comparability.
4. Detect terminals as keypoints to support transistors and op-amps, and add AC sources with transient simulation.
5. Improve crossover handling with local line-direction analysis at wire crossings.
6. Extend the text path to the sentence structures it still misses ("across both of them", "through", a parallel group before "in series with"), add named nodes for circuits that are not series-parallel, and write a larger hand-written test set.

References

[1] L. W. Nagel and D. O. Pederson, "SPICE (Simulation Program with Integrated Circuit Emphasis)," Memo. ERL-M382, Electronics Research Laboratory, University of California, Berkeley, 1973.

[2] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[3] F. Thoma, J. Bayer, Y. Li and A. Dengel, "A public ground-truth dataset for handwritten circuit diagram images," in *Proc. Int. Conf. Document Analysis and Recognition (ICDAR) Workshops*, Springer, 2021, pp. 20–27.

[4] P. Kumar and P. Shinde, "CircuitVision" source code, experiment notebook and result files. [Online]. Available: https://github.com/sudo-pranshu/CircuitVision

[5] B. Edwards and V. Chandran, "Machine recognition of hand-drawn circuit diagrams," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2000.

[6] M. Moetesum, S. W. Younus, M. A. Warsi and I. Siddiqi, "Segmentation and recognition of electronic components in hand-drawn circuit diagrams," *EAI Endorsed Trans. Scalable Information Systems*, vol. 5, no. 16, pp. 1–5, Apr. 2018, doi: 10.4108/eai.13-4-2018.154478.

[7] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE CVPR*, 2005, pp. 886–893.

[8] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[9] M. D. Patare and M. S. Joshi, "Hand-drawn digital logic circuit component recognition using SVM," *Int. J. Computer Applications*, 2016.

[10] J. Bayer, A. K. Roy and A. Dengel, "Instance segmentation based graph extraction for handwritten circuit diagram images," in *Proc. 12th Int. Conf. Pattern Recognition Applications and Methods (ICPRAM)*, 2023. arXiv:2301.03155.

[11] J. Bayer, L. van Waveren and A. Dengel, "Modular graph extraction for handwritten circuit diagram images," arXiv:2402.11093, 2024.

[12] CGHD dataset, Zenodo, record 17469897 (version 16), 2025.

[13] J. Bayer et al., "CGHD: Circuit Graph Hand-Drawn dataset," Zenodo, record 14042961 (version 14), file cghd-zenodo-14.zip, MD5 ab1cde6feb5edaafbde711cd2059a2f4. Licence CC BY 3.0.

[14] R. R. Reddy and M. R. Panicker, "Hand-drawn electrical circuit recognition using object detection and node recognition," arXiv:2106.11559, 2021.

[15] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proc. IEEE*, vol. 77, no. 2, pp. 257–286, 1989.

[16] A. J. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," *IEEE Trans. Information Theory*, vol. 13, no. 2, pp. 260–269, 1967.

[17] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. draft, 2024. [Online]. Available: https://web.stanford.edu/~jurafsky/slp3/

[18] J. Earley, "An efficient context-free parsing algorithm," *Communications of the ACM*, vol. 13, no. 2, pp. 94–102, 1970.

[19] G. Jocher, A. Chaurasia and J. Qiu, "Ultralytics YOLO," version 8.4.184, 2023–2026. [Online]. Available: https://github.com/ultralytics/ultralytics

[20] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. European Conf. Computer Vision (ECCV)*, 2014, pp. 740–755.

[21] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Trans. Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

[22] R. E. Tarjan, "Efficiency of a good but not linear set union algorithm," *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, 1975.

[23] Y. Baek, B. Lee, D. Han, S. Yun and H. Lee, "Character region awareness for text detection," in *Proc. IEEE CVPR*, 2019, pp. 9365–9374.

[24] B. Shi, X. Bai and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.

[25] JaidedAI, "EasyOCR," version 1.7.2. [Online]. Available: https://github.com/JaidedAI/EasyOCR

[26] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.

[27] H. Vogt, G. Atkinson and P. Nenzi, *Ngspice User's Manual*, version 42. [Online]. Available: https://ngspice.sourceforge.io

[28] A. Abid et al., "Gradio: Hassle-free sharing and testing of ML models in the wild," arXiv:1906.02569, 2019.

Appendix A: Important Source Code

Only key excerpts are included; the complete, tested source code is in the project repository [4].

A.1 Paper-relative binarisation (preprocess.py)

```
def binarize(img, kernel=None, min_contrast=12):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) if img.ndim == 3 else img
    gray = cv2.GaussianBlur(gray, (5, 5), 0)
    k = kernel or max(15, (min(gray.shape[:2]) // 15) | 1)
    paper = cv2.morphologyEx(gray, cv2.MORPH_CLOSE,
                             cv2.getStructuringElement(cv2.MORPH_RECT, (k, k)))
    dark = (255.0 * cv2.subtract(paper, gray)
            / np.maximum(paper, 1)).astype(np.uint8)
    cand = dark[dark > min_contrast]
    thr = min_contrast
    if cand.size > 100:
        otsu, _ = cv2.threshold(cand.reshape(1, -1), 0, 255,
                                cv2.THRESH_BINARY + cv2.THRESH_OTSU)
        thr = max(min_contrast, int(otsu))
    mask = np.where(dark > thr, 255, 0).astype(np.uint8)
    return cv2.morphologyEx(mask, cv2.MORPH_OPEN, np.ones((2, 2), np.uint8))
```

A.2 Viterbi decoding (nlp/hmm.py)

```
def tag(self, words):
    obs = [self.w_index[self._norm(w)] for w in words]
    n, T = len(obs), len(self.tags)
    score = np.empty((n, T)); back = np.zeros((n, T), int)
    score[0] = self.log_start + self.log_emit[:, obs[0]]
    for i in range(1, n):
        cand = score[i - 1][:, None] + self.log_trans      # prev x cur
        back[i] = cand.argmax(0)
        score[i] = cand.max(0) + self.log_emit[:, obs[i]]
    path = [int(score[-1].argmax())]
    for i in range(n - 1, 0, -1):
        path.append(int(back[i, path[-1]]))
    return [self.tags[t] for t in reversed(path)]
```

A.3 Netlist grammar (grammar.py)

```
GRAMMAR = {
  "S":       [("TITLE", "EOL", "BODY", "ENDLINE"), ("TITLE", "EOL", "ENDLINE")],
  "TITLE":   [("comment",)],
  "BODY":    [("LINE",), ("LINE", "BODY")],
  "LINE":    [("STMT", "EOL")],
  "STMT":    [("R",), ("C",), ("L",), ("V",), ("D",), ("MODEL",),
              ("dotop",), ("comment",)],
  "R":       [("rname", "node", "node", "value")],      # C and L likewise
  "V":       [("vname", "node", "node", "SRC")],
  "SRC":     [("value",), ("dc", "value")],
  "D":       [("dname", "node", "node", "ident")],
  "MODEL":   [("dotmodel", "ident", "ident")],
  "ENDLINE": [("dotend", "EOL")],
}
```

Appendix B: Configuration Details

The baseline training configuration is listed in Table 5 and the class mapping in Table 4. Polarity follows the Gate 1 decision, which is also the code default (`left`, counter-clockwise); it can be overridden with two environment variables: `CIRCUITVISION_PLUS_AT_0` (left, right, top or bottom) and `CIRCUITVISION_CGHD_CLOCKWISE` (0 or 1). The detector confidence used by the downstream pipeline is 0.35; the qualitative error analysis uses 0.25 at IoU 0.5.

To reproduce the real experiment: open `notebooks/train_colab.ipynb` in Google Colab with a T4 GPU and choose Run all. The notebook downloads CGHD release 14, verifies its MD5, audits it, stops at both gates for human approval, trains, evaluates and writes an evidence ZIP. Command-line equivalents are documented in `EXPERIMENTS.md` in the repository.

Appendix C: Sample Dataset Record

Excerpt from a real CGHD annotation (`drafter_13/C145_D1_P1.xml`, CC BY 3.0), kept in the repository as a regression-test fixture. Note that `<rotation>` is nested inside `<bndbox>`, and that the drawing contains out-of-scope symbols (lamp, switch).

```
<annotation>
  <filename>C145_D1_P1.jpg</filename>
  <size><width>1536</width><height>2049</height><depth>3</depth></size>
  <object>
    <name>lamp</name>
    <bndbox>
      <xmin>674</xmin><ymin>703</ymin><xmax>826</xmax><ymax>845</ymax>
    </bndbox>
  </object>
  <object>
    <name>voltage.battery</name>
    <bndbox>
      <xmin>399</xmin><ymin>897</ymin><xmax>588</xmax><ymax>967</ymax>
      <rotation>269</rotation>
    </bndbox>
  </object>
  <object>
    <name>switch</name> ...
</annotation>
```

Appendix D: Test Cases

Table 18**:** Automated regression tests (129 in total)

Test file	Tests	Covers
test_topology.py	54	Wire tracing: scale, grid, shadow, pen gaps, junctions, crossovers
test_cghd_format.py	14	Real XML, rotation, mirror, EXIF, placeholder sizes, voltage.dc, ground
test_text2netlist.py	8	Tagger and parser; 330R, "a pair of"; frozen set; error categories
test_compare.py	5	Isomorphism and operating-point comparison
test_grammar.py	5	Earley validator and semantic checks
test_audit_images.py	4	Header image sizes equal full decodes, all EXIF orientations
test_dataset.py	4	CGHD-to-YOLO conversion counts and labels
test_detector_eval.py	4	Detector evaluation harness
test_evaluate.py	6	Downstream harness; scoring against reference netlists
test_references.py	13	Every drafter-12 reference netlist passes the grammar check
test_orientation.py	4	Rotation convention and HOG + SVM learning
test_pipeline.py	4	End to end on drawn fixtures; crossovers; trailing-R values
test_simulate.py	2	ngspice operating point
test_colab_support.py	2	Training resume detection; label-check sampling